

# ШПАРГАЛКА: DOM И СОБЫТИЯ

- `document.querySelector(".class")` - находит первый элемент по CSS-селектору
  - `document.querySelectorAll(".class")` - находит все элементы, возвращает список (NodeList)
  - `document.getElementById("id")` - находит элемент по id (без решётки в скобках)
  - `document.getElementsByClassName("class")` - находит все элементы с классом (без точки)
- 
- `nodeList.forEach(el => {...})` - переберёт каждый элемент из `querySelectorAll` по очереди
  - NodeList — это не совсем массив, но `forEach` у него работает так же, как у обычного массива
  - HTMLCollection (результат `getElementsByClassName`) — не имеет `forEach` напрямую, его сначала нужно превратить в массив
- 
- `element.style.свойство = "значение"` - меняет inline-стиль элемента напрямую из JS
  - CSS-свойства с дефисом пишутся в camelCase: `background-color` → `backgroundColor`
  - `element.style.цвет` принимает строку — как в CSS: `"#378ADD"`, `"red"`, `"rgb(0,0,0)"`

## Подсказка

Если нужно изменить стиль одного конкретного элемента — какой метод поиска вернёт именно один элемент? А если нужно применить стиль ко всем подходящим элементам сразу — какой метод и какая конструкция для перебора вам понадобятся?

- `element.addEventListener("событие", функция)` - вешает обработчик на событие
  - `click` - клик мышью
  - `mouseover` - наведение курсора на элемент
  - `mouseout` - курсор ушёл с элемента
  - `submit` - отправка формы
  - `keydown / keyup` - нажатие / отпускание клавиши
- 
- Функция-обработчик получает аргумент — объект события, обычно называют `event` или `e`
  - `event.target` - ссылка на элемент, на котором произошло событие
  - `event.target.textContent` - текстовое содержимое этого элемента

- `event.preventDefault()` - отменяет стандартное поведение (например переход по ссылке)

### Подсказка

Чтобы вывести в консоль реальный текст заголовка (а не написанный руками) — подумайте, что окажется внутри объекта события, если навести курсор именно на этот заголовок, и как достать его текстовое содержимое.

- `element.classList.add("class")` - добавляет класс
- `element.classList.remove("class")` - убирает класс
- `element.classList.toggle("class")` - добавляет класс, если его нет; убирает, если уже есть
- `element.classList.contains("class")` - проверяет, есть ли класс (вернёт true/false)

### Подсказка

Toggle сам решает — добавить класс или убрать. Значит, чтобы кнопка меняла цвет туда-обратно, вам нужно продумать два состояния в CSS: обычное и с модификатором. Toggle просто переключает, какое из них сейчас видно — вся логика цвета должна быть уже прописана в стилях, а не в JS.

- 
- `window.open("url", "_blank")` - открывает адрес в новой вкладке
  - `window.location.href = "url"` - переходит по адресу в текущей вкладке
  - `console.log(значение)` - выводит значение в консоль разработчика (не видно пользователю сайта)
  - `alert("текст")` - показывает всплывающее окно поверх страницы (видно пользователю)